COMS 4720/5720 Recitation

Uninformed Search (Week 4)

Hyungtak Lee
Slides adapted from Dr. Yan-Bin Jia

Department of Computer Science
Iowa State University
Ames, IA 50011
htlee@iastate.edu

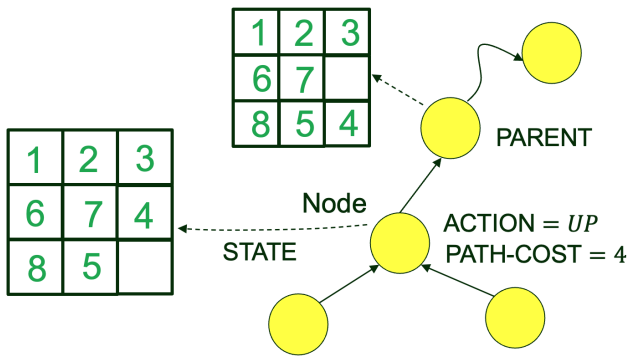
Feb 11, 2026

Outline

Search Tree

Choose a node n with minimum value of some evaluation function $f(n)$.

- Return it its state is a goal state.
- Otherwise generate child nodes and add them to the frontier.



Components of a node:

- State: unvisited, alive, and dead.
- Parent: node that generated this node.
- Action: action applied to Parent.State to generate this node.
- Path-Cost: total cost from root to this node following the path.

More on Date Structure

Frontier: the set of generated leaf nodes that are unexpanded and eligible for expansion.

- Is-Empty()
- Pop()
- Insert(n)
- Resolve-Duplicate(n, *frontier*, *reached*)

The key difference between search algorithm lies in how they order the frontier and how they handle duplicated states.

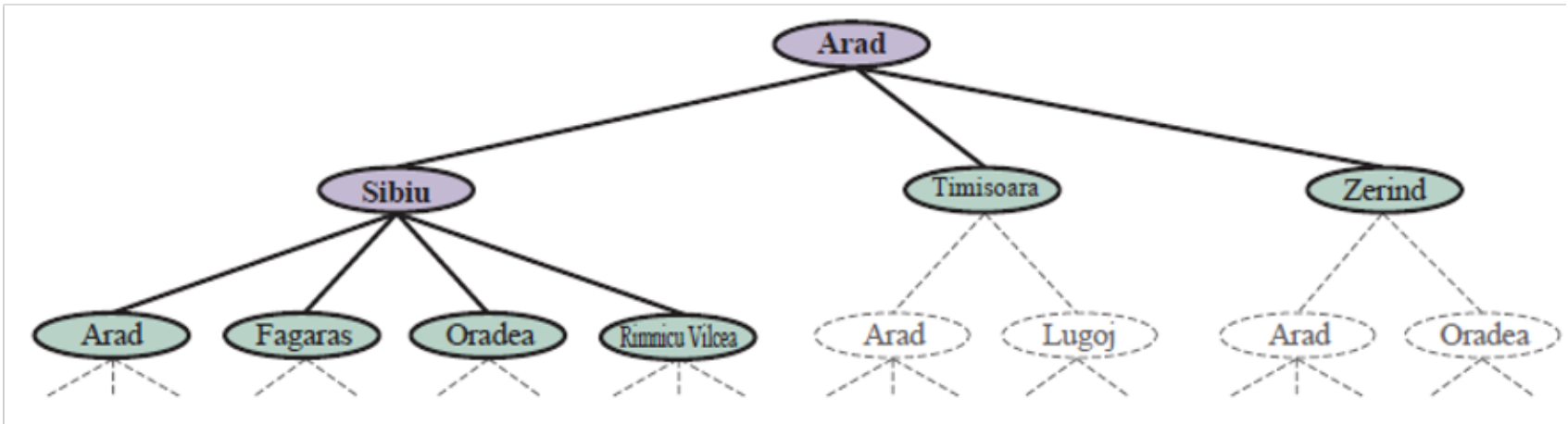
- *frontier*: FIFO for BFS and LIFO for DFS.
- Resolve-Duplicate

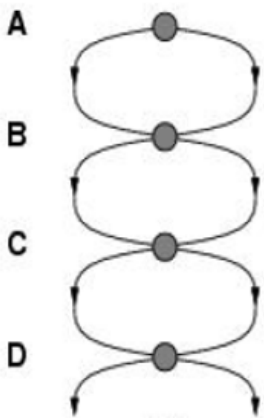
A General Template for Forward Search Algorithm

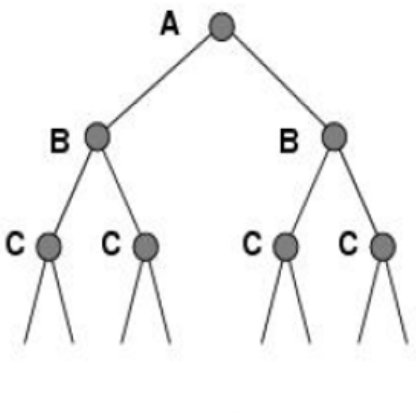
Algorithm 1 A general template for forward search algorithms

```
1:  Function SEARCH(problem, frontier) return a solution node or failure
2:      node ← NODE(STATE = problem.INITIAL(), PARENT = ∅, ACTION = ∅, PATH-COST = 0)
3:      if problem.IS-GOAL(node.STATE) then return node
4:      frontier.INSERT(node)
5:      reached ← a lookup table, with one entry with key problem.INITIAL and value node
6:      expanded ← an empty lookup table
7:      while not frontier.IS-EMPTY() do
8:          n ← frontier.POP()
9:          if problem.IS-GOAL(n.STATE) then return n
10:         s = n.STATE
11:         if s is not in expanded then
12:             expanded[s] ← n
13:             for all a in problem.ACTIONS(s) do
14:                 s' ← problem.RESULT(s, a)
15:                 cost ← n.PATH-COST + problem.ACTION-COST(s, a, s')
16:                 n' ← NODE(STATE = s, PARENT = n, ACTION = a, PATH-COST = cost)
17:                 if s' is not in reached then
18:                     reached[s'] ← n'
19:                     frontier.INSERT(n')
20:                 else if s' is not in expanded then
21:                     frontier.RESOLVE-DUPLICATE(n', frontier, reached)
```

Repeated States







1) Failure to detect repeated states can turn a solvable problem into an unsolvable one.
2) Keep only the best path to each state.
3) Resolve-Duplicate

Performance Measures

- Completeness: Is the algorithm guaranteed to find a solution whenever one exists?
  (The state space may be infinite!)
- Cost optimality: Does it find a solution with the lowest path cost of all solutions?
- Time complexity: Physical time or the number of states and actions.
- Space complexity: Memory needed for the search.

Breadth-First Search (BFS)

- Uninformed search: No clue about how close a state is to the goal.
- Can be viewed as a specific instantiation of best-first search: $f(n)$ is simply the depth of the node n.
- *frontier* can be implemented as a FIFO queue.
- Properties of BFS:
  - Systematic search: It will find a solution in finite time if one exists, both for finite and infinite state space.
  - Complete even when the state space is infinite (the branching factor is finite).
  - Always finds a solution with a minimum number of actions.

Breadth-First Search (BFS) (Cont'd)

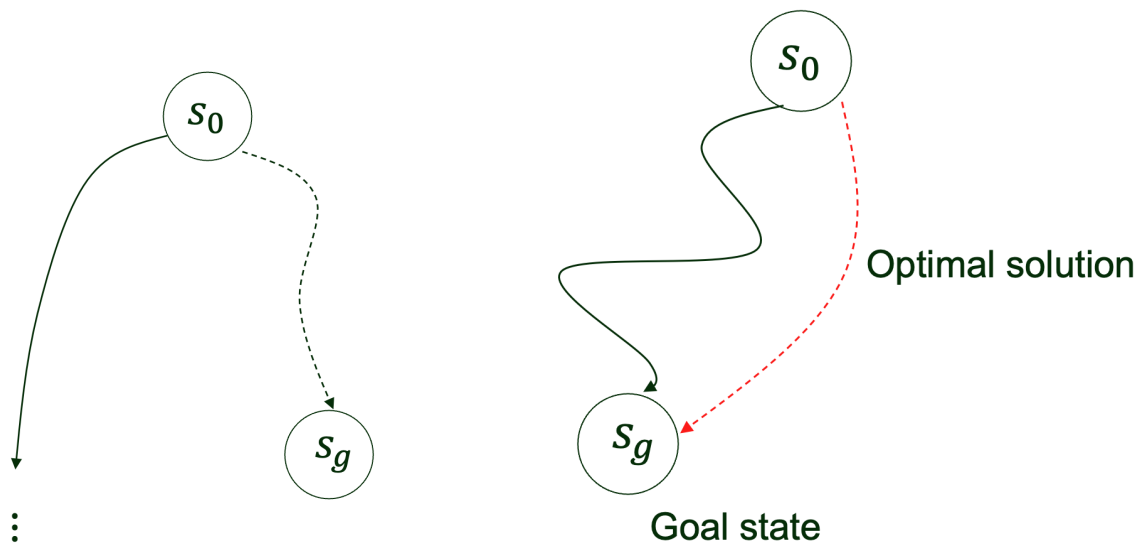
- BFS on a uniform tree where every node has b successors where b is a branching factor.
  - b nodes at depth 1 generated by the root.
  - Each node at depth 1 generates b nodes, i.e., b^2 nodes at depth 2.
  - And so on.
- Assume solution on depth d: the number of nodes be

$$1 + b + \cdots + b^d = \frac{b^{d+1} - 1}{b - 1} = O(b^d). \tag{1}$$

- Time and space complexities: $O(b^d)$.
- Only small search problems are solvable due to exponential time complexity.

Depth-First Search (DFS)

- Uninformed search: No clue about how close a state is to the goal.
- Can be viewed as a specific instantiation of best-first search: $f(n)$ is the negative depth of the node n.
- *frontier* can be implemented as a LIFO queue (stack).
- Properties of DFS:
  - Systemic only for finite state space; not systemic for infinite state space (down an infinite corridor).
  - Not cost-optimal



Depth-First Search (DFS) (Cont'd)

- Time complexity $O(b^m)$, where b is the branching factor and m is the maximum depth of the search tree.
- Space complexity $O(bm)$, if cycles exist.

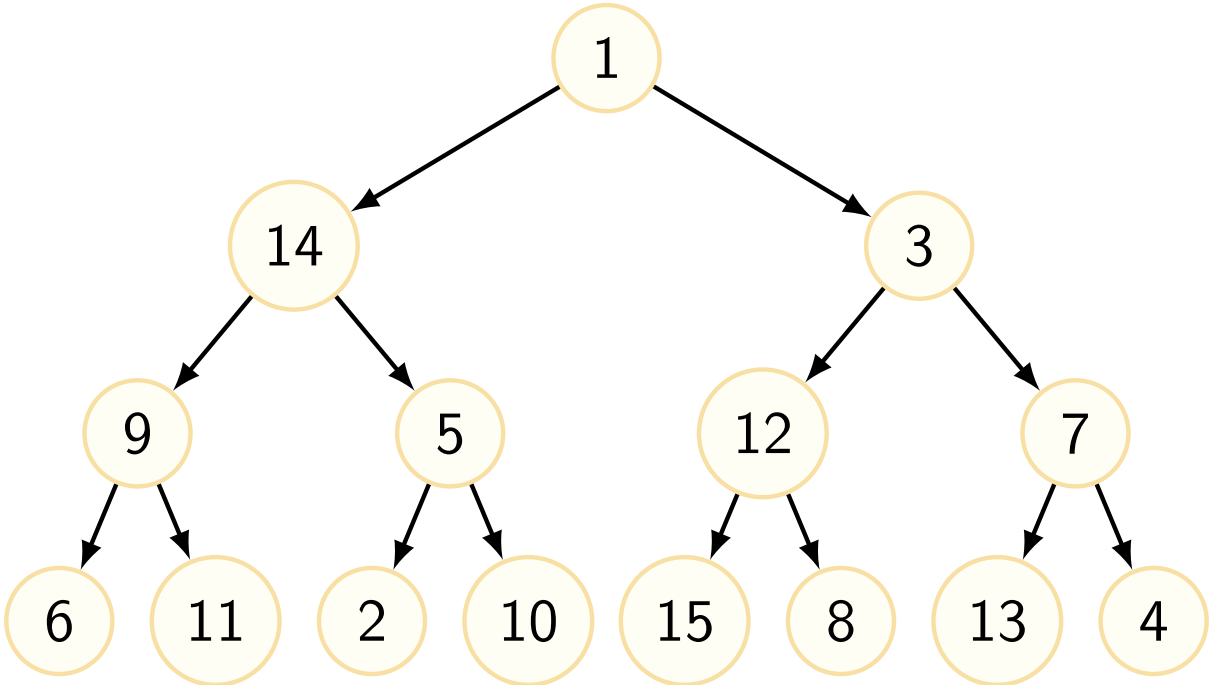
Depth-Limited Search

- To avoid DFS from exploring an infinite path, add a depth limit l.
- All nodes at depth l are considered dead ends even if they have successors.
- Time complexity: $O(b^l)$.
- Space complexity: $O(bl)$.
- Performance sensitive to the choice for l.

Iterative Deepening Search (IDS)

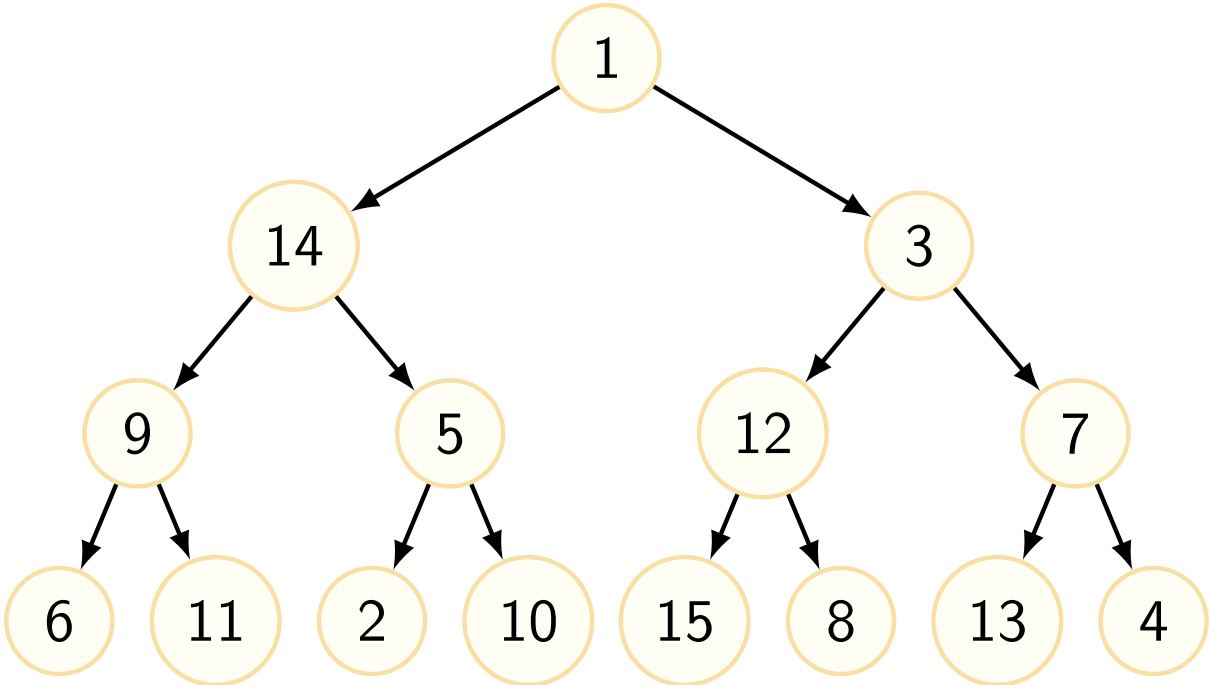
- Pick a good value for l by trying all values: 0, 1, 2, and so on.
- Repeatedly runs depth-limited search with increasing l until a solution is found.
- Combines the benefits of DFS and BFS:
  - Completeness and optimality of BFS.
  - Small memory requirement of DFS (only the current search path is stored).
- If goal state is found at depth d:
  - Time complexity: $O(b^d)$ (same as BFS)
  - Space complexity: $O(bd)$ (same as DFS)
- If goal state is found in a finite state space:
  - Time complexity: $O(b^m)$, where m is the maximum depth explored.
  - Space complexity: $O(bm)$.

Example: Tree search



Suppose the goal state is 10.

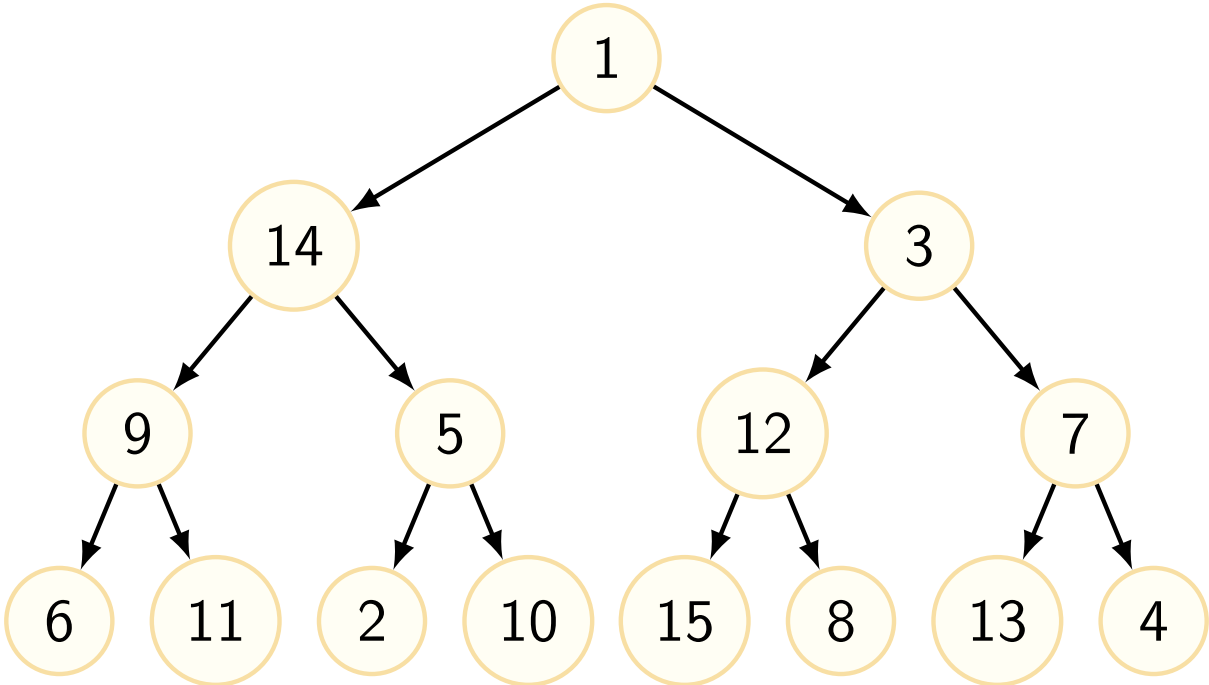
Example: BFS



Breadth-First Search (BFS):

1, 14, 3, 9, 5, 12, 7, 6, 11, 2, 10

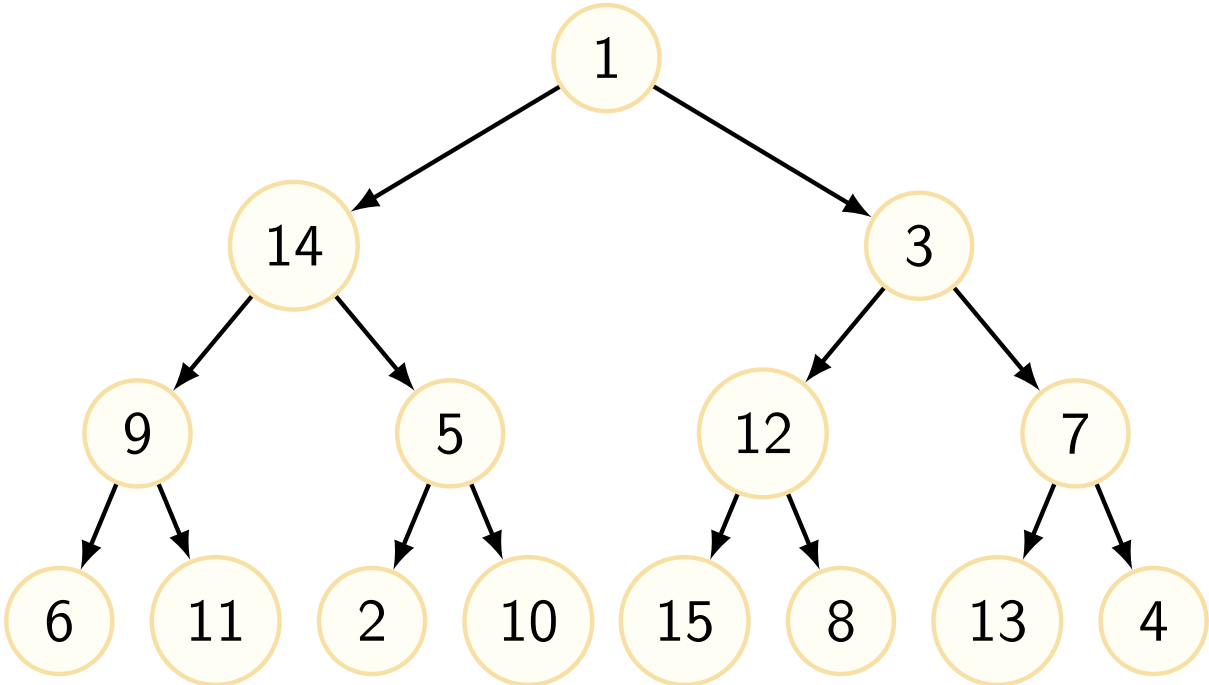
Example: DFS



Depth-First Search (DFS):

1, 14, 9, 6, 11, 5, 2, 10

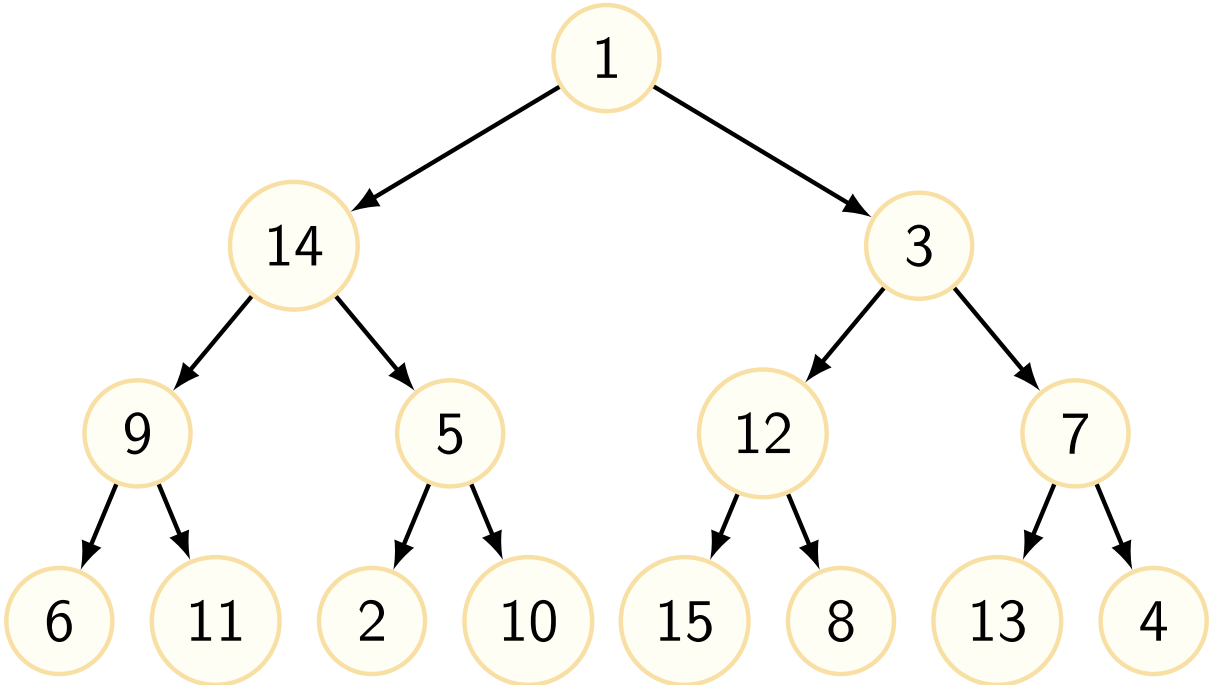
Example: Depth-limited search



Depth-Limited Search (Limit 3):

1, 14, 9, 6, 11, 5, 2, 10

Example: Iterative Deepening Search (IDS)



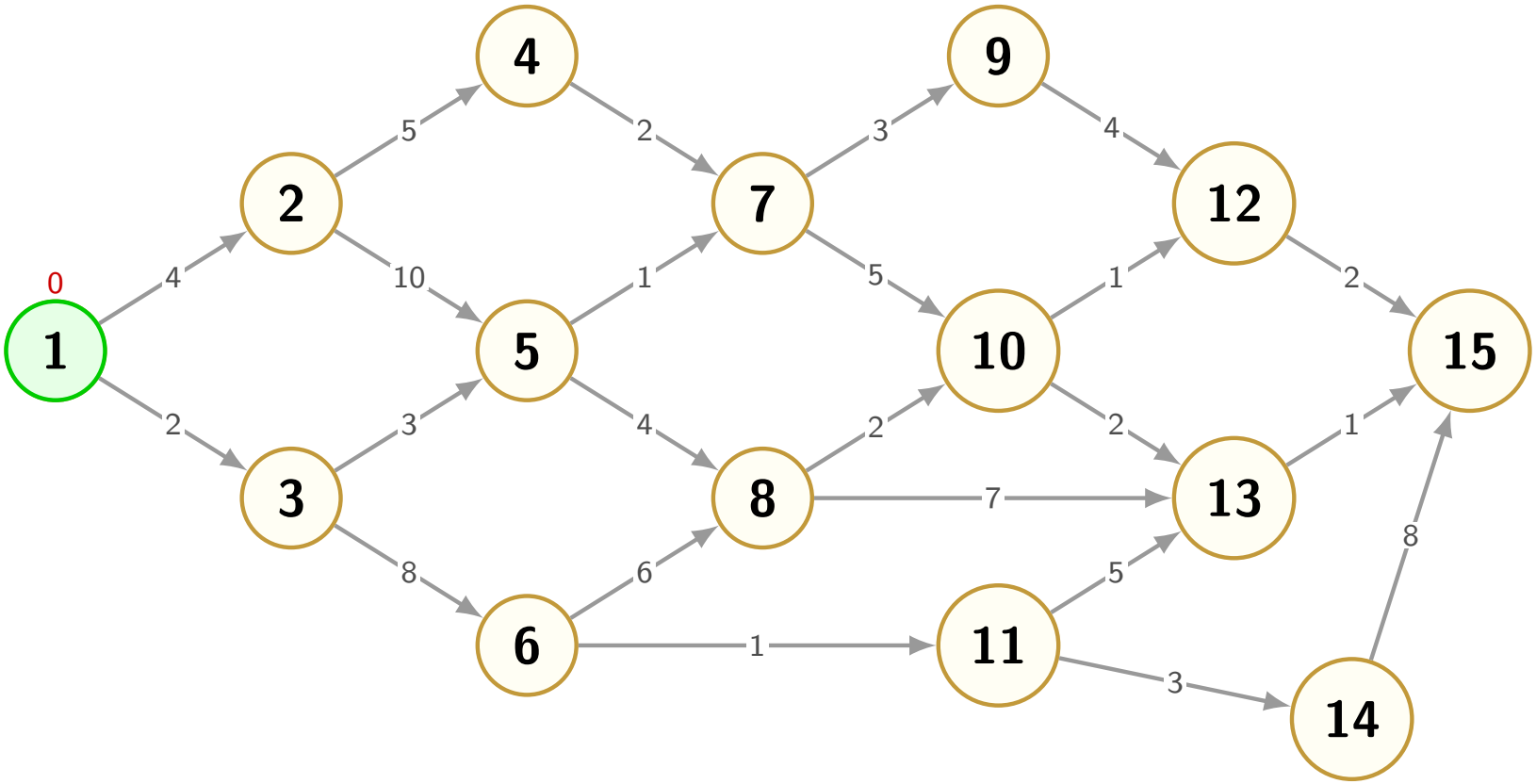
Iterative Deepening Search (IDS):

- **Depth 0:** 1
- **Depth 1:** 1, 14, 3
- **Depth 2:** 1, 14, 9, 5, 3, 12, 7
- **Depth 3:** 1, 14, 9, 6, 11, 5, 2, 10

Dijkstra's Algorithm (Uniform-Cost Search)

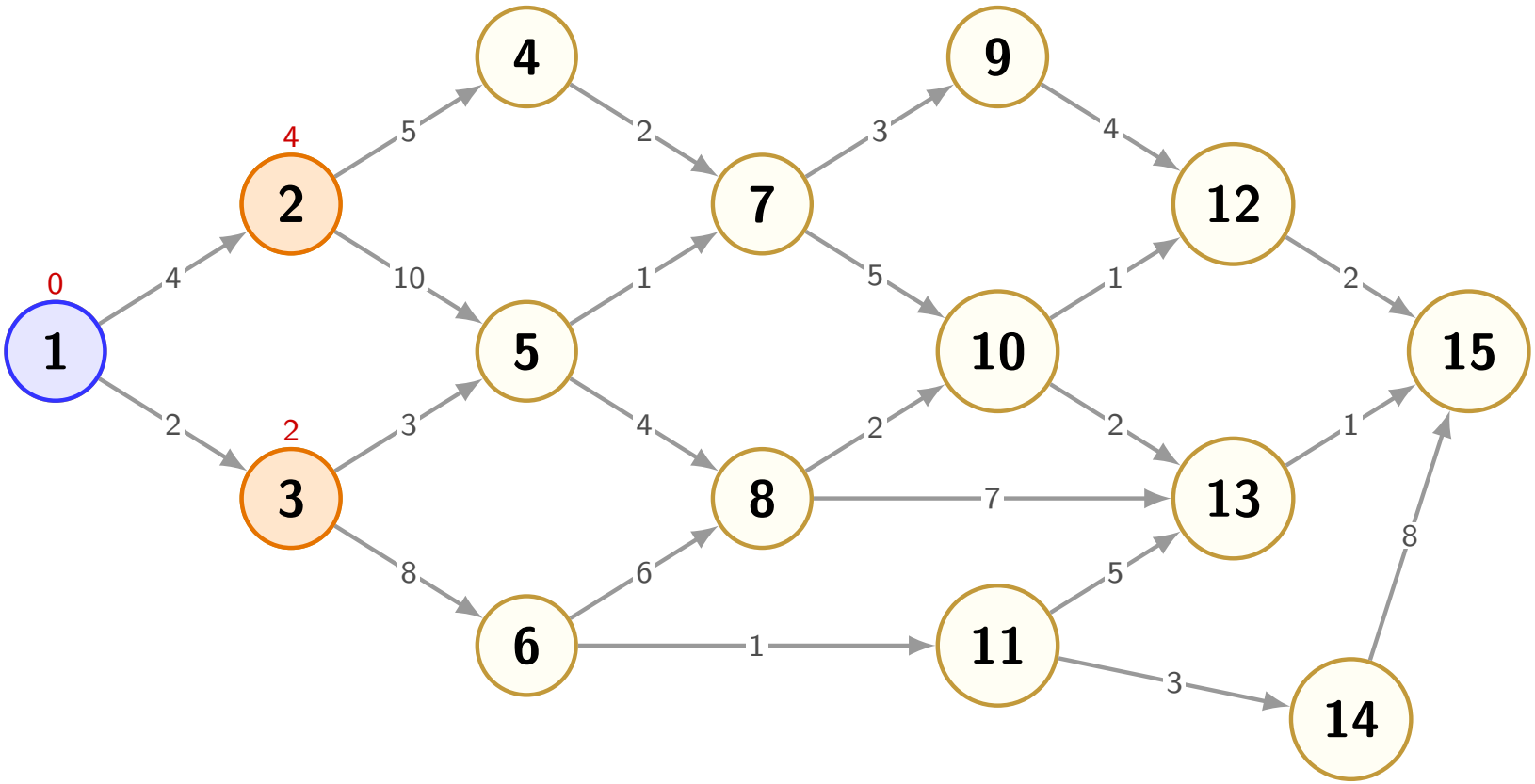
- Uninformed search: No clue about how close a state is to the goal.
- Can be viewed as a specific instantiation of best-first search: $f(n) = n.\textsc{Path-Cost}$.
- *frontier* is implemented as a priority queue sorted with the smallest $\textsc{Path-Cost}$.

Example: Graph



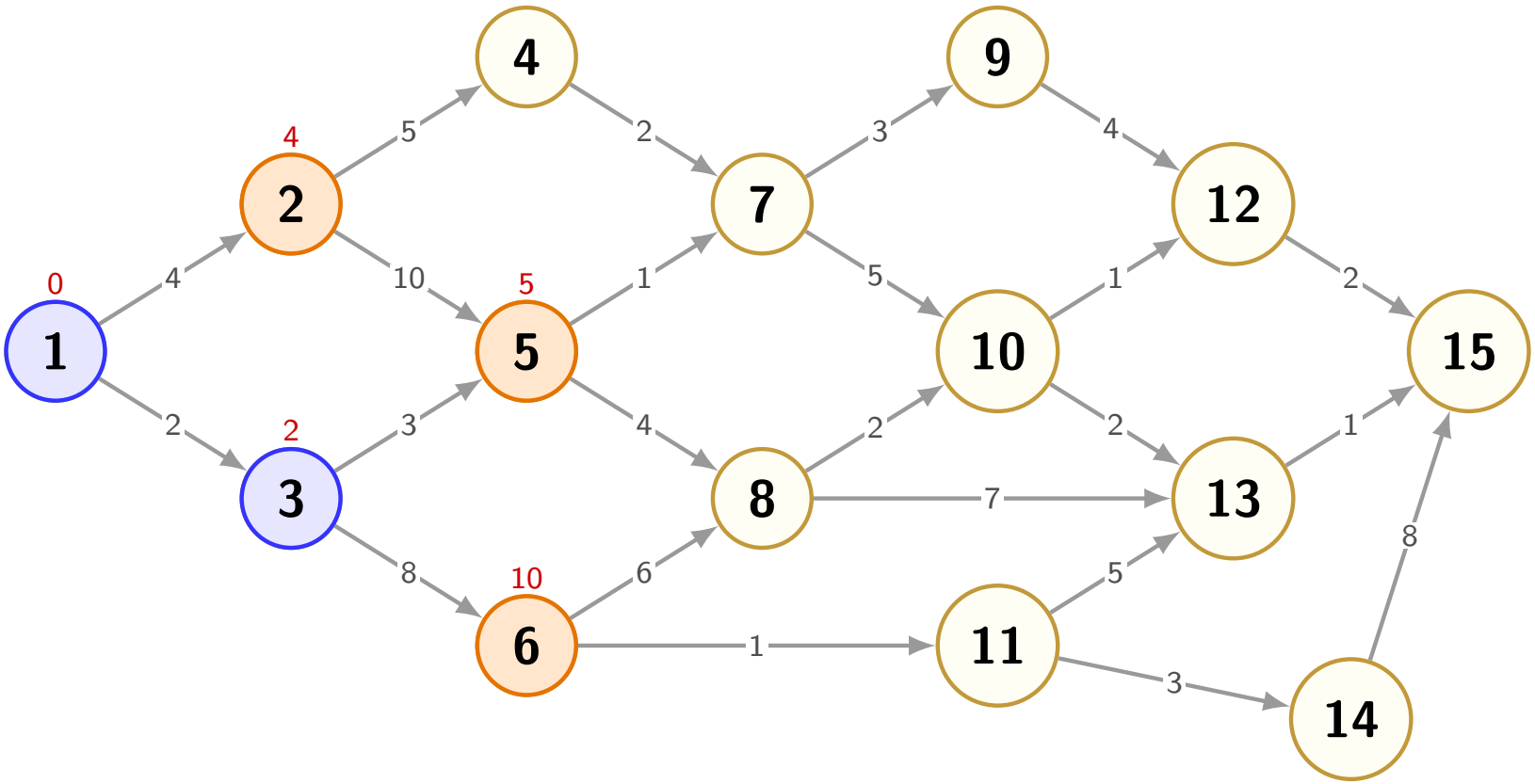
Current: **1**. Frontier: [ ].

Example: Graph



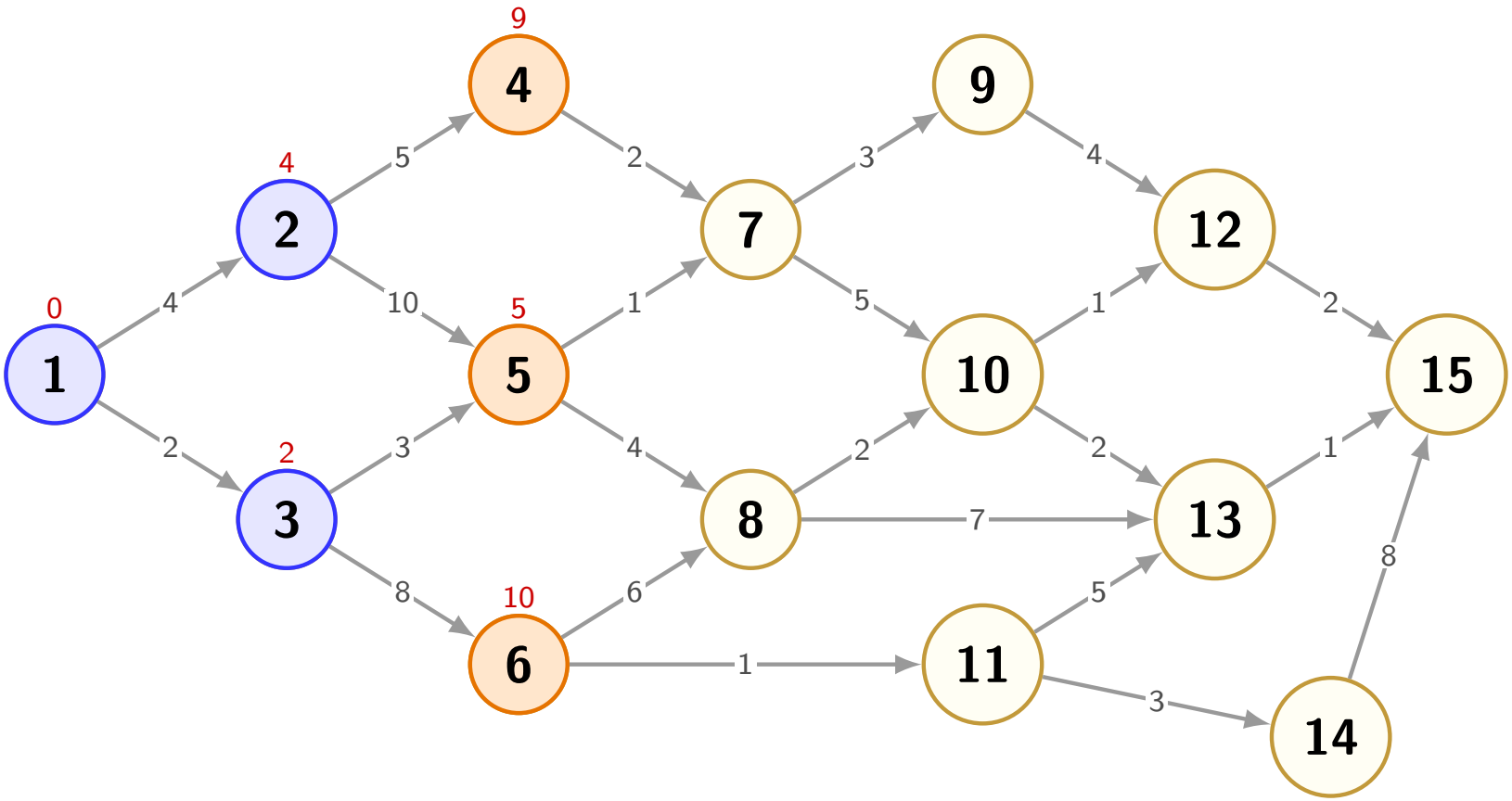
Expand 1. Add neighbors to Frontier: $2(p = 4)$, $3(p = 2)$.
Next pick: **3**.

Example: Graph



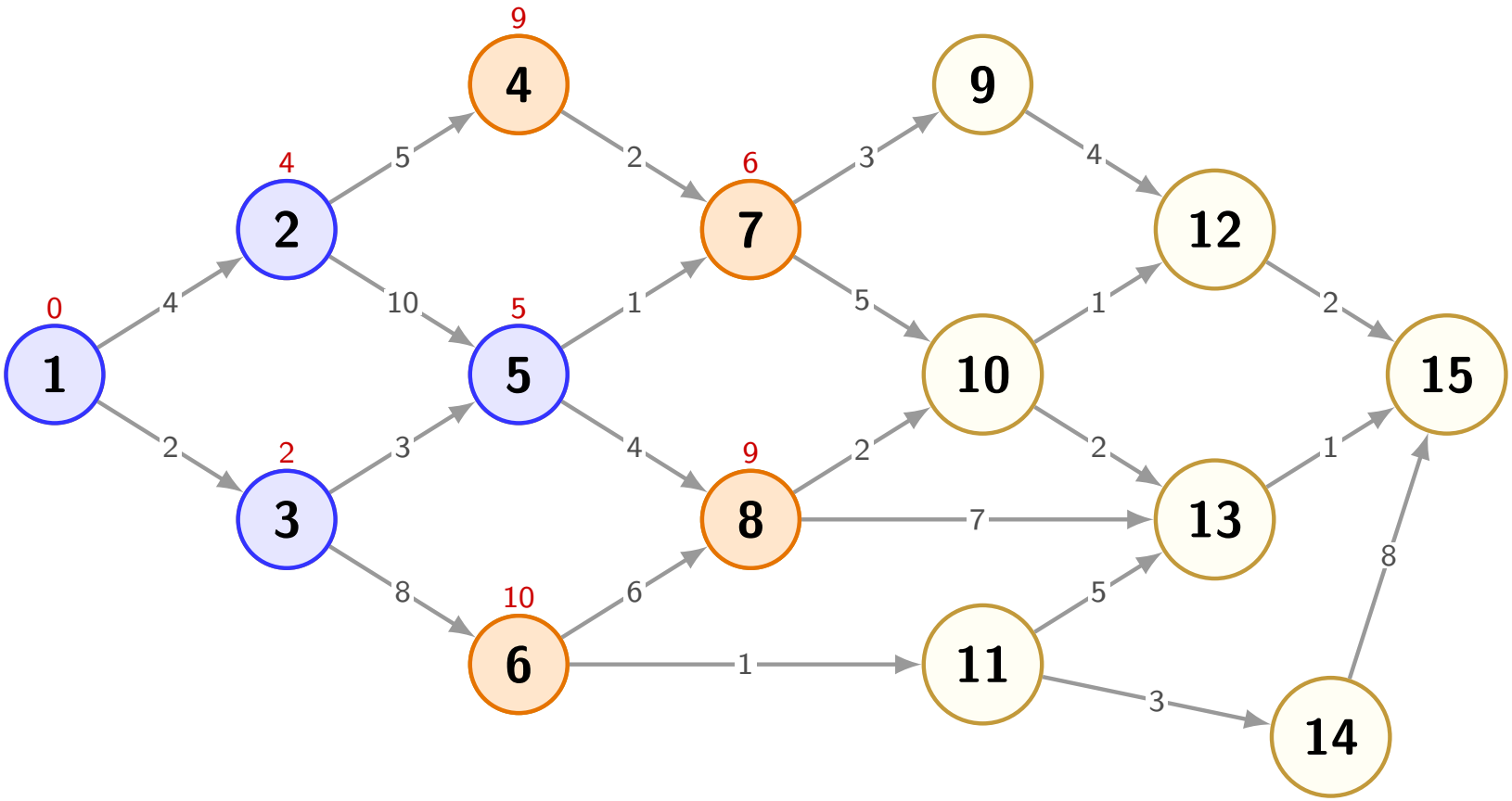
Expand 3. Update neighbors: $5(p = 5)$, $6(p = 10)$.
Frontier: [2:4, 5:5, 6:10]. Next pick: **2**.

Example: Graph



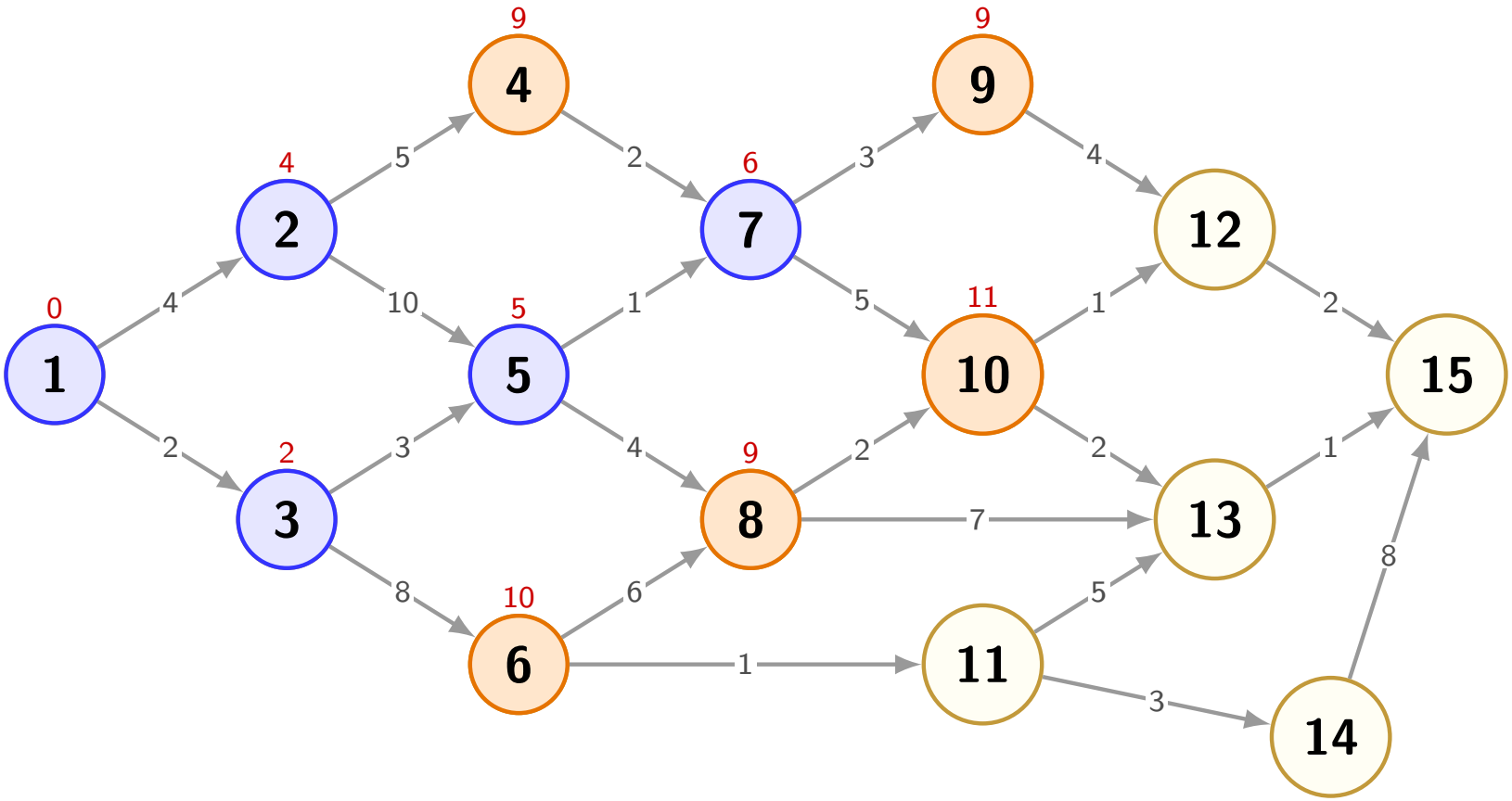
Expand 2. Add 4($d = 9$).
Frontier: [5:5, 4:9, 6:10]. Next pick: **5**.

Example: Graph



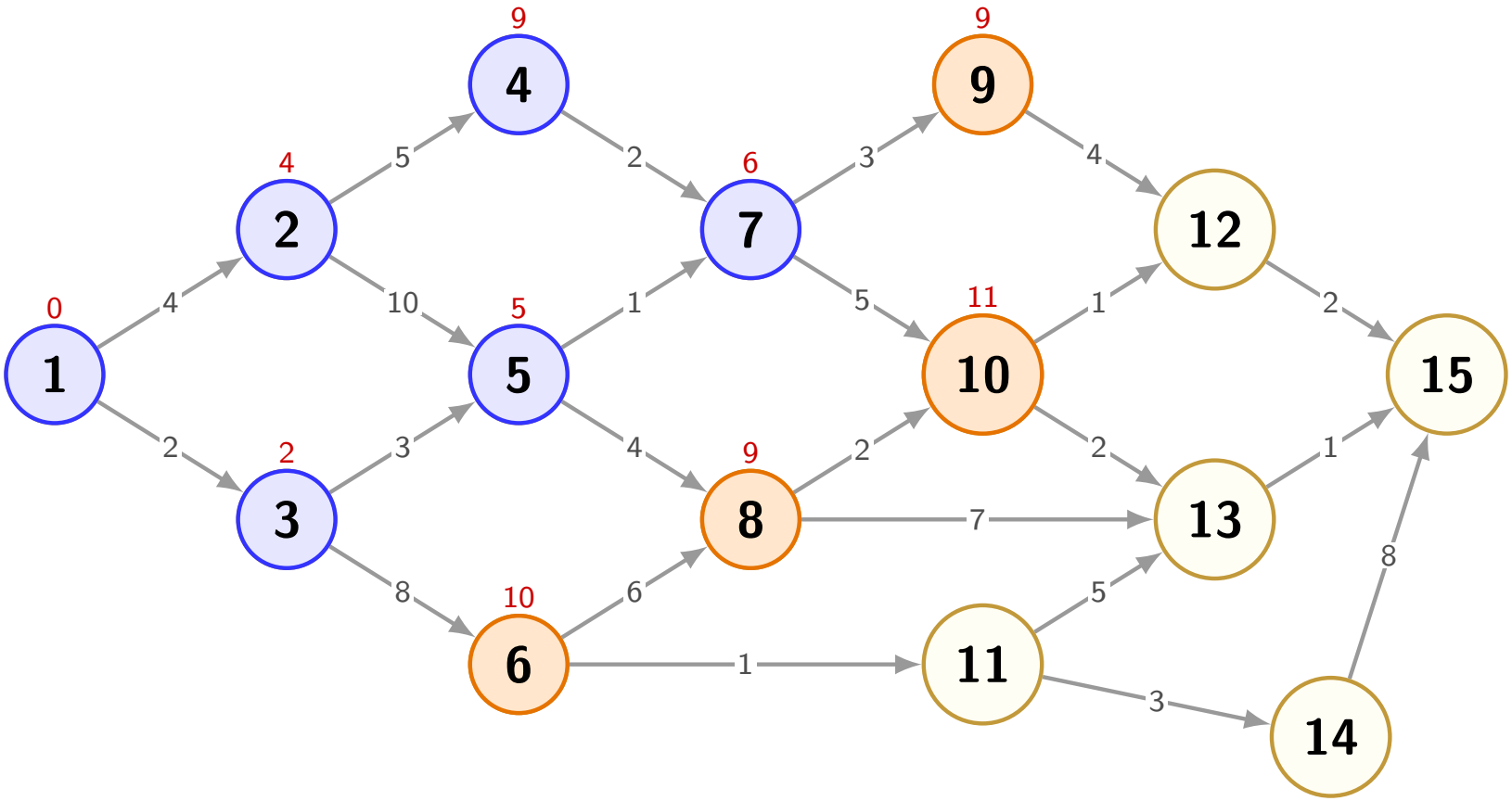
Expand 5. Update $7(p = 6)$, $8(p = 9)$.
Frontier: [7:6, 4:9, 8:9, 6:10]. Next pick: **7**.

Example: Graph



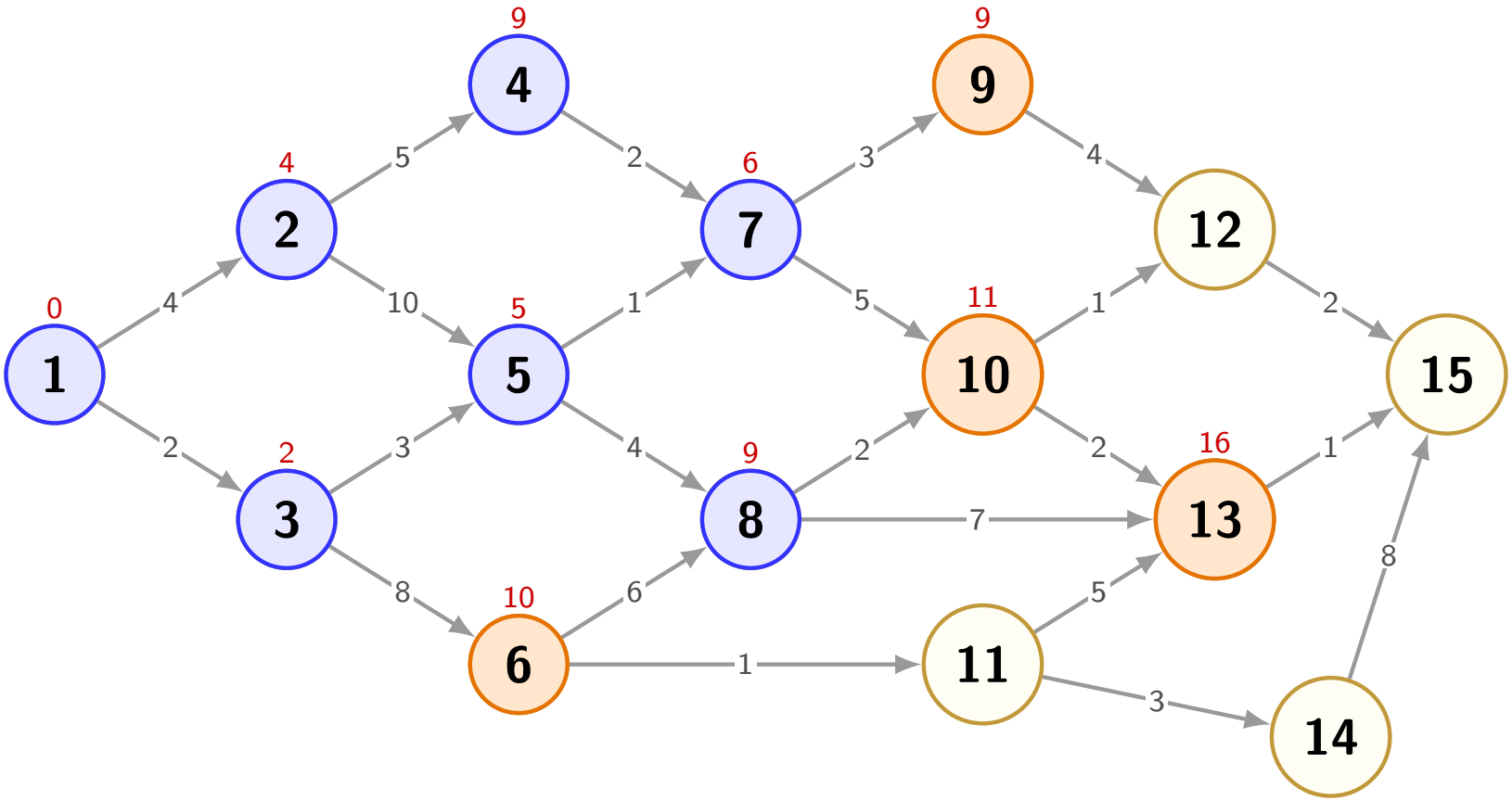
Expand 7. Add 9($p = 9$), 10($p = 11$).
Frontier: [4:9, 8:9, 9:9, 6:10...]. Next pick: **4**.

Example: Graph



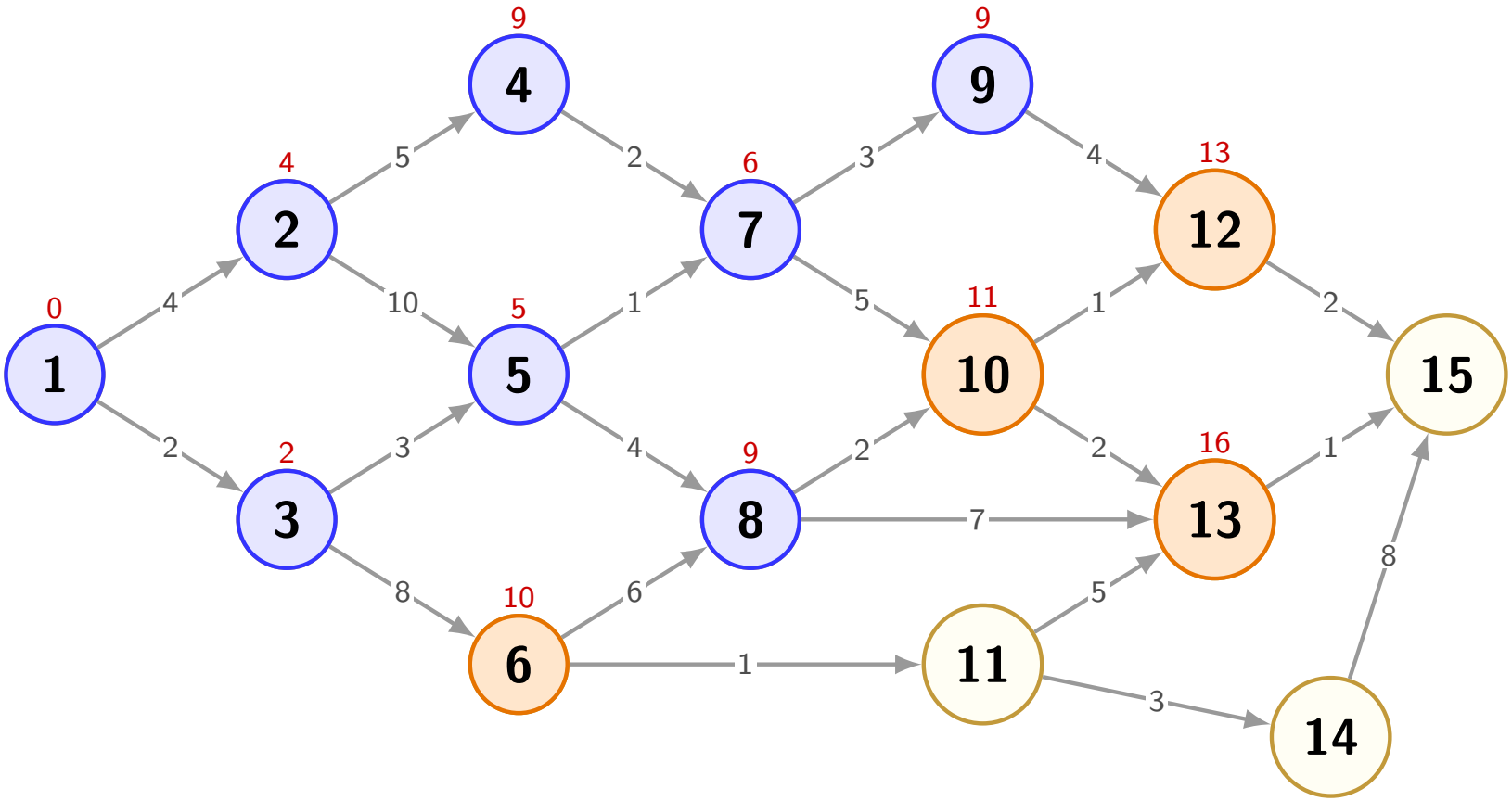
Expand 4. No new updates.
Frontier: [8:9, 9:9, 6:10, 10:11]. Next pick: **8**.

Example: Graph



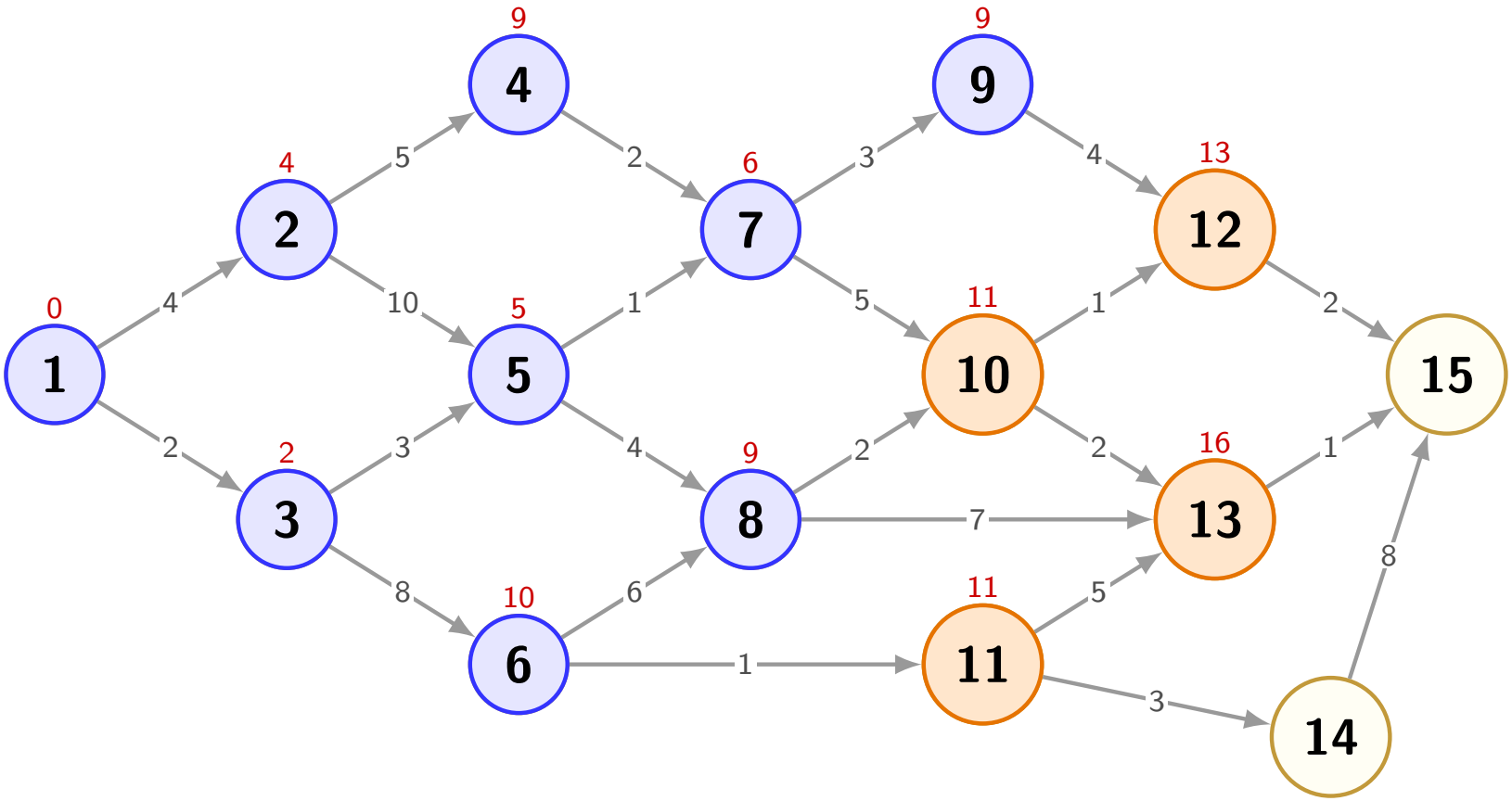
Expand 8. Add 13($p = 16$). 10 unchanged.
Frontier: [9:9, 6:10, 10:11, 13:16]. Next pick: **9**.

Example: Graph



Expand 9. Add 12($p = 13$).
Frontier: [6:10, 10:11, 12:13, 13:16]. Next pick: **6**.

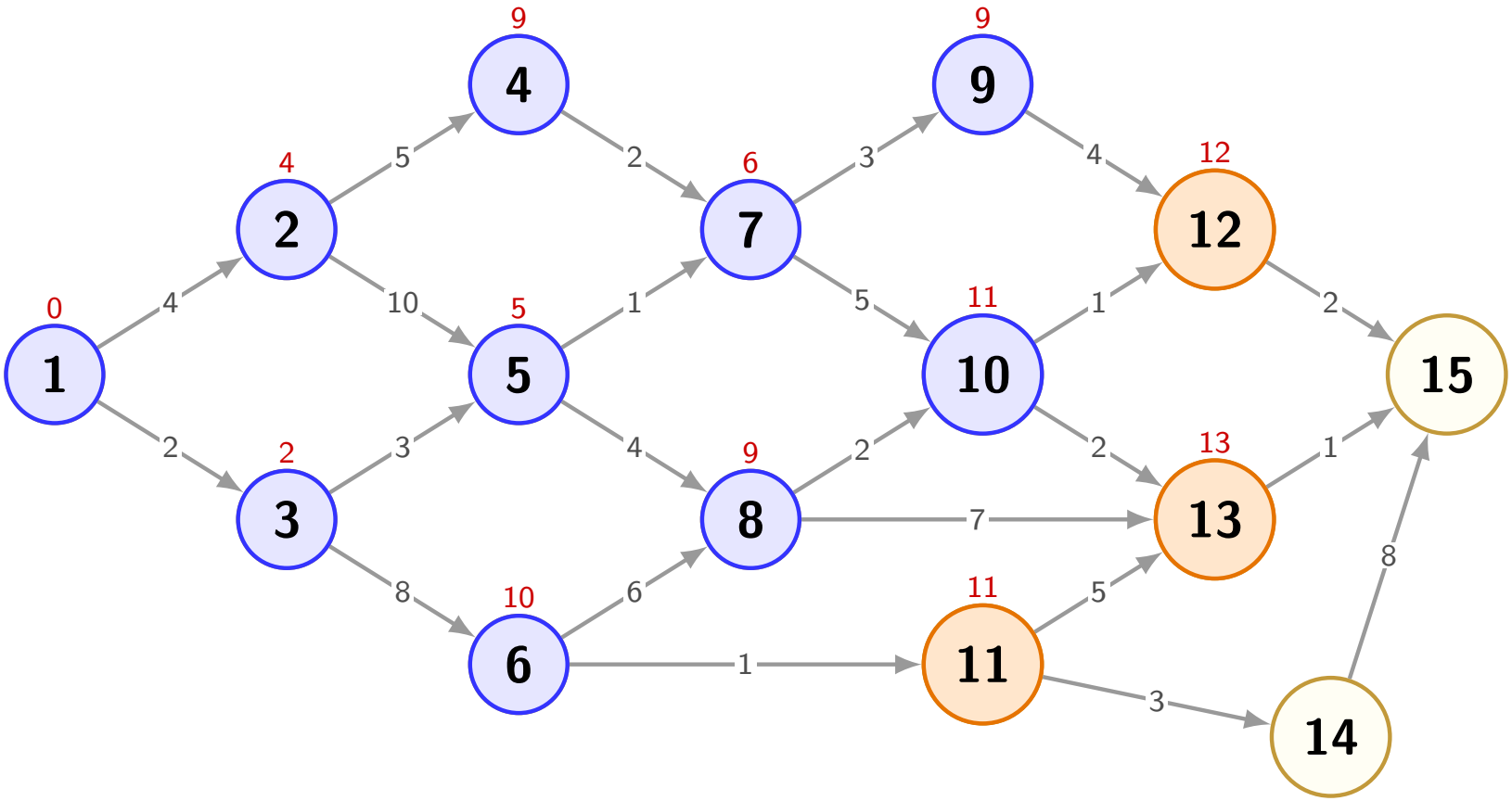
Example: Graph



Expand 6. Add 11($p = 11$).

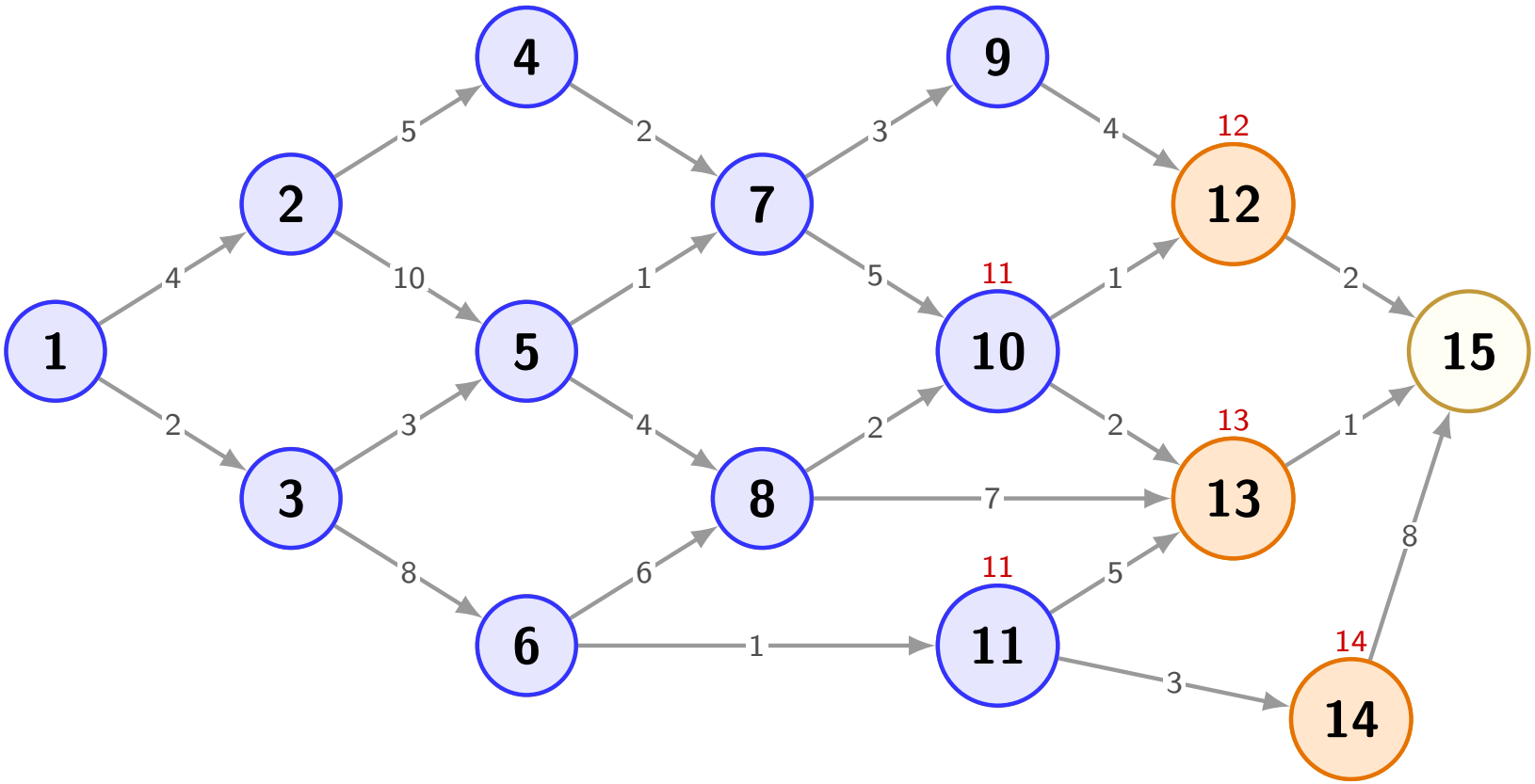
Frontier: [10:11, 11:11, 12:13, 13:16]. Next pick: **10**.

Example: Graph



Expand 10. Update 12($p = 12$), 13($p = 13$).
Frontier: [11:11, 12:12, 13:13]. Next pick: **11**.

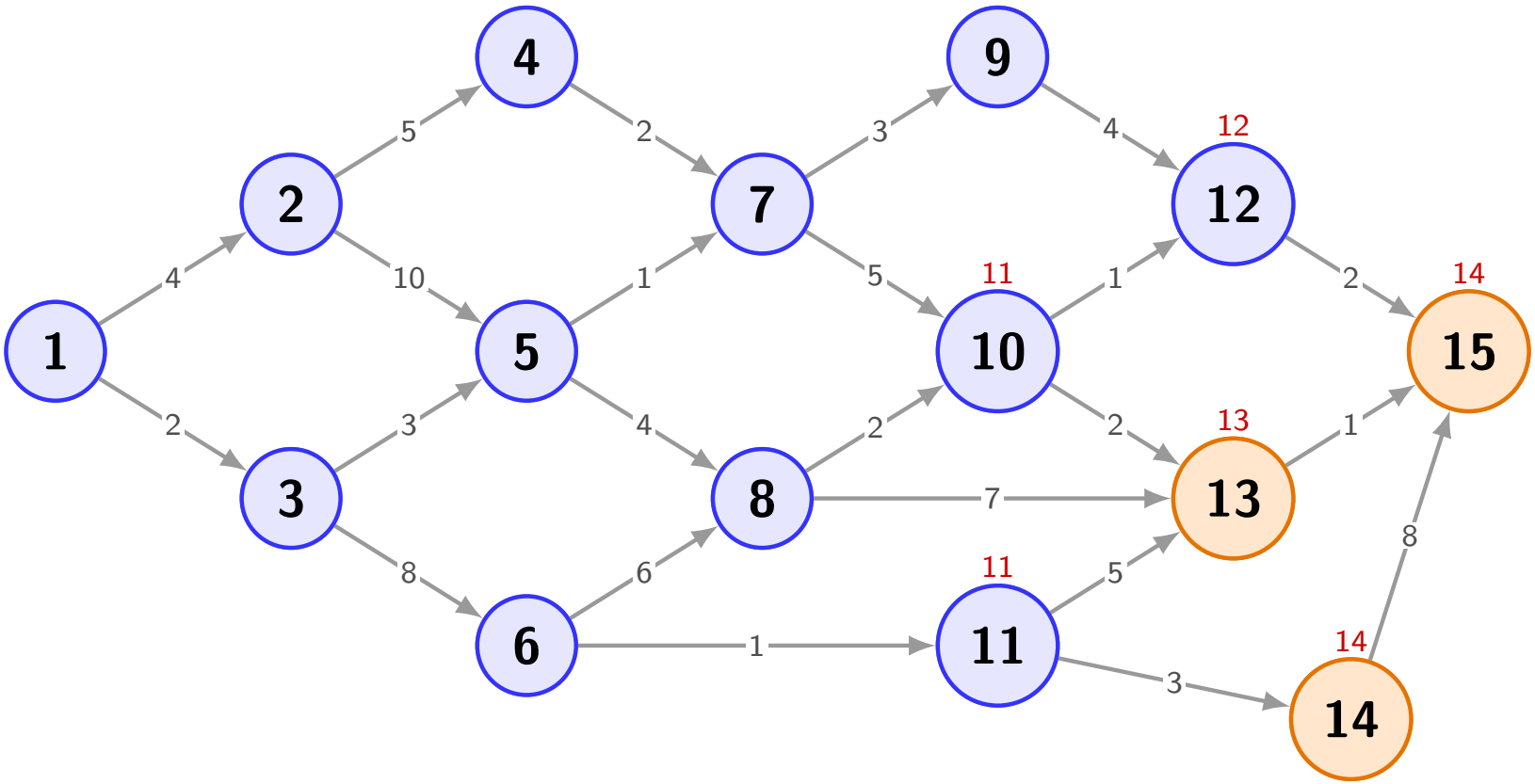
Example: Graph



Expand 11. Add 14($p = 14$).

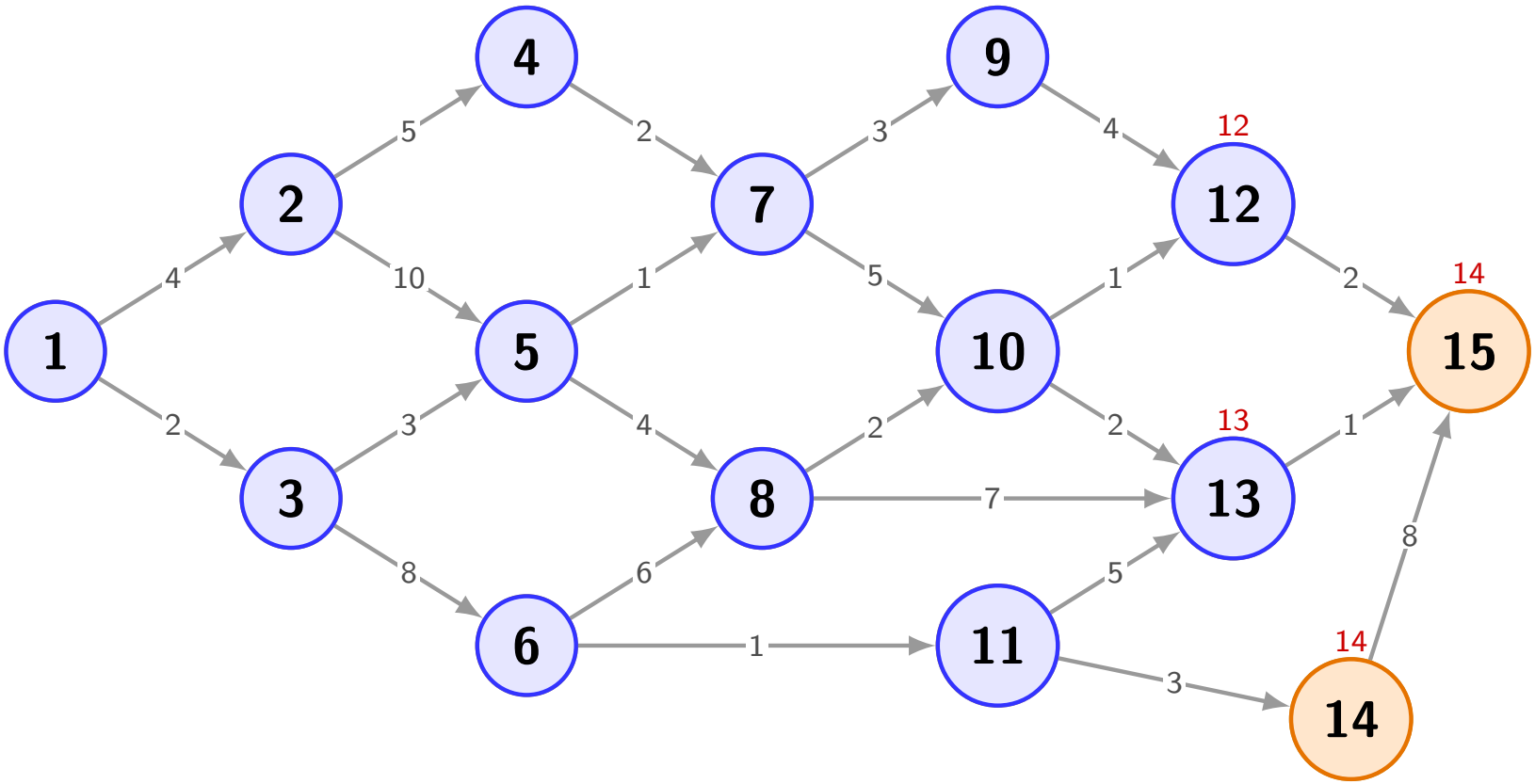
Frontier: [12:12, 13:13, 14:14]. Next pick: **12**.

Example: Graph



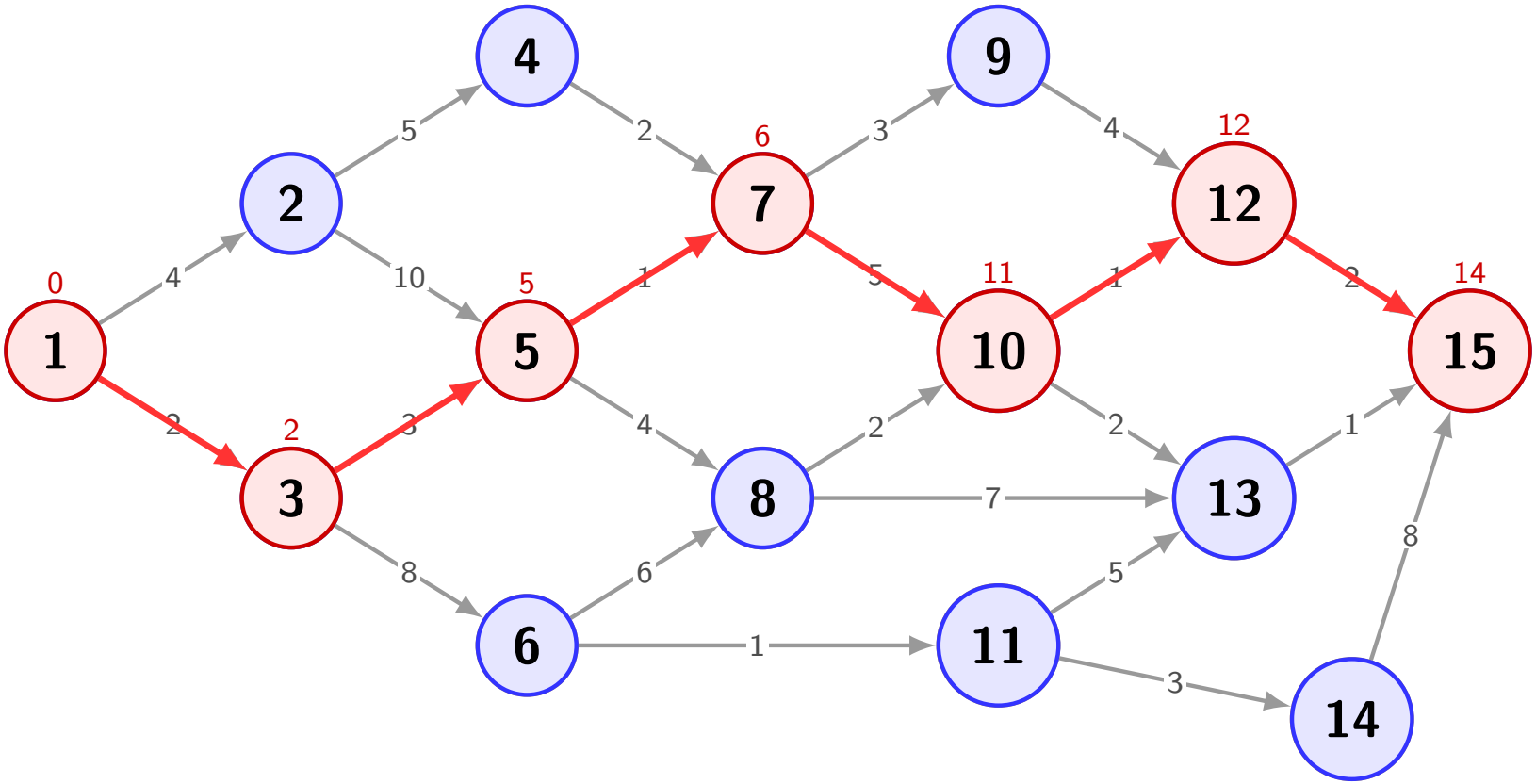
Expand 12. Update 15($p = 14$).
Frontier: [13:13, 15:14, 14:14]. Next pick: **13**.

Example: Graph



Expand 13. No better path to 15 found.
Frontier: [15:14, 14:14]. Next pick: **15 (Goal)**.

Example: Graph



Shortest Path Found!

Cost: 14

Path: $1 \to 3 \to 5 \to 7 \to 10 \to 12 \to 15$ (Cost: 14)